pystarlight

Rubén García-Benito

September 29, 2016

Contents

1 Reading *STARLIGHT* output with *pystarlight* **2**
1.1 Single values . . . 2
1.2 Array-like values . . . 4
1.2.1 `Spectra` Table . . . 4
1.2.2 `Population` Table . . . 8
1.3 Weighted average properties . . . 16
1.3.1 Fraction of young, intermediate and old populations: x_Y, x_I, x_O . . . 18
1.4 Estimating the Star Formation Rate (SFR) . . . 20
1.5 StarlightUtils . . . 22

Reading *STARLIGHT* output with *pystarlight*

pystarlight is a `python` module for reading the `STARLIGHT` ouput, making very easy the work with this stellar fitting code results.

Let's start by opening a file.

pystarlight depends on **ATpy** and **asciitable** modules, so make sure you have them installed.

ATpy is a high-level Python package providing a way to manipulate tables of astronomical data in a uniform way.

Much of **ATpy**'s functionality has now been incorporated into **Astropy**, but the `TableSet` is still to be implemented, so we have to rely on **ATpy**.

Other that that, **ATpy** works pretty much as **astropy.tables**, with some sintaxis slightly different.

A `TableSet` instance contains a Python list of individual instances of the `Table` class.

The advantage of using a `TableSet` instead of building a Python list of `Table` instances manually is that **ATpy** allows reading and writing of groups of tables to file formats that support it (e.g. FITS and VO table files or SQL databases).

```
In [1]: import pystarlight.io.starlighttable #@UnusedImport
        import atpy

        slfile = '../data/spectra/SL/0414.51901.393.cxt.sc4.C11.im.CCM.BS'
        tsl = atpy.TableSet(slfile, type='starlightv4')
```

`import pystarlight.io.starlighttable` is just an import of the format table type for **STARLIGHT** (`type='starlightv4'`), so **ATpy** can understand the format of the output.

Then, `tsl` is a `TableSet` object with several tables.

1.1 Single values

All the float, integer or string values with only one item will be stored in the `.keywords` dictionary attribute.

This is a dictionary like the `.meta` **astropy.tables** attribute. In fact, when you save an **ATpy** `TableSet` instance as a FITS file, the `.keywords` is save in the header with its keywords.

Let's have a look a the `keyword` dictionary keywords:

```
In [2]: print (tsl.keywords.keys())
```

```
['Nl_eff', 'file_version', 'Flux_tot', 'iFitPowerLaw', 'q_norm', 'NEX0s_base', 'llow_no
```

You might notice that these are just those individuals values at the beggining of the file:

Accesing these values is very easy, it is a dictionary:

```
################################################################
## OUTPUT of StarlightChains_v04.for    [Cid@UFSC - 01/Apr/2007] ##
################################################################

## Some input info
0414.51901.393.cxt                      [arq_obs]
Base.BC03.S                             [arq_base]
Mask.0414.51901.393.cxt.sc1.CRAP.gm.BN  [arq_masks]
StCv04.C11.config                       [arq_config]
  150                                   [N_base]
    0                                   [N_YAV_components = # of components with extra extinction!]
    0                                   [i_FitPowerLaw (1/0 = Yes/No)]
-9.99                                   [alpha_PowerLaw]
CCM                                     [red_law_option]
 1.458241                               [q_norm = A(l_norm)/A(V)]

## (Re)Sampling Parameters
 3400.00                                [l_ini (A)]
 8900.00                                [l_fin (A)]
    1.00                                [dl    (A)]     (OBS: dl_cushion =    50.00 A)

## Normalization info
 4020.00                                [l_norm (A) - for base]
 4010.00                                [llow_norm (A) - window for f_obs]
 4060.00                                [lupp_norm (A) - window for f_obs]
1.655215E+01                            [fobs_norm (in input units)]

## S/N
 4730.00                                [llow_SN (A) - window for S/N]
 4780.00                                [lupp_SN (A) - window for S/N]
  24.912                                [S/N in S/N window]
  32.739                                [S/N in norm. window]
  16.546                                [S/N_err in S/N window]
  21.744                                [S/N_err in norm. window]
  1.0000                                [fscale_chi2]
  ...
```

```
In [3]: print (tsl.keywords['l_norm'])
```

```
4020.0
```

The astrophysical results that you will probably use more frequently are extinction (`A_V`), velocity shift (`v_0`) and velocity dispersion (`v_d`)

Exercise: print the results of extinction, velocity shift and velocity dispersion

```
In [4]: print tsl.keywords['A_V']
        print tsl.keywords['v_0']
        print tsl.keywords['v_d']
```

```
0.2684
-21.08
43.39
```

Also, a very important variable in this dictionary is `Mcor`, which allows you to get an estimation of the stellar mass of your spectra (in solar units) if you know the distance and the flux calibration of the spectra is reliable:

$$M_* = M_{cor_tot} \times flux_units \times 4\pi d^2 \times (3.826 \times 10^{33})^{-1}$$

Exercise: if we assume a distance of 126 Mpc, and `flux_units` are 1e-17, calculate the mass of the galaxy in $M_\odot$

M_* is the total mass in $M_\odot$, provided the input flux is in ergs/cm^2/s/Å and your SSP in $L_\odot$ Å^{-1} $M_\odot^{-1}$

$L_\odot = 3.826 \times 10^{33}$

Hint: use **astropy.constants**

```
In [5]: from astropy.constants import L_sun
        import astropy.units as u
        from math import pi

        distance = 126.
        flux_units = 1e-17 * u.erg / u.cm**2 / u.s

        dist_cm = (distance * u.Mpc).to(u.cm)
        mcor_tot = tsl.keywords['Mcor_tot']
        mass = mcor_tot * flux_units *\
                    4 * pi * dist_cm**2 / L_sun.cgs

        print mass
        print '%.4g M_sun' % mass.value
```

```
124363113.048
1.244e+08 M_sun
```

1.2 Array-like values

Array-like values are saved as columns of a `Table`.

There are 2 tables: one with the stellar populations results (`population`) and other with the input and output sythetic spectra (`spectra`).

1.2.1 `Spectra` Table

The `spectra` table contains 4 columns:

```
In [6]: print (tsl.spectra.columns)
```

```
{
l_obs : type=>f8,
f_obs : type=>f8,
f_syn : type=>f8,
f_wei : type=>f8
}
```

- `l_obs`: Wavelength (rest frame, as original input)
- `f_obs`: Input original spectra
- `f_syn`: The synthetic fitted spectra
- `f_wei`: The weights array

The spectra are given in units of `fobs_norm`, which you can access in the `.keywords` dictionary.

Exercice

Always look at the fit!!

Make a figure that contains two subplots. The main subplot (bigger) should contain the input spectra and the synthetic spectra and evaluate the fit. The characteristics should be:

- Input and synthetic spectra in normalized units (recommended range between 0 ~ 2.5)
- Different colors for the input and synthetic spectra
- Labels and legend ("Observed" and "Synthetic")
- A title, with the name of the spectra file (taken from the **pystarlight** output).

For the second subplot, plot the residual spectrum (f_obs - f_syn) in normalized units. The optimal `y` limits for the residual is around (-0.2,0.5)

Somewhere in the plot, you can add some other important astrophysical results:

- Extinction (`A_V`)
- Velocity shift (`v_0`)
- Velocity dispersion (`v_d`)
- *mass* of the object (as option if the distance is provided: use **astropy.constants**)

and quality of the plots:

- χ^2
- `adev`

Try to do it as a function like, so you can built an executable file (use **argparse**) to open and plot any **STARLIGHT** output from the command line.

You can add options like plotting the main subplot (input and synthetic) in normalized or absolute units, distance to calculate the mass, ...

** Solution **

```
In [7]: %matplotlib inline
        from matplotlib.ticker import AutoMinorLocator
        from astropy.constants import L_sun
        import matplotlib.pyplot as plt
        from matplotlib import gridspec
        import astropy.units as u
        from math import pi
```

```
distance = 128.1 # Mpc (fake)
flux_units = 1.0e-17

fig = plt.figure(figsize=(10,8))
gs = gridspec.GridSpec(2, 1, height_ratios=[3, 1])
# No space between subplots
gs.update(left=0.05, bottom=0.05, right=0.98, hspace=0.0)

# Subplot with input and synthetic spectra
ax1 = plt.subplot(gs[0])

fobs_norm = tsl.keywords['fobs_norm']
ax1.plot(tsl.spectra.l_obs, tsl.spectra.f_obs, label='Observed')
ax1.plot(tsl.spectra.l_obs, tsl.spectra.f_syn, label='Synthetic')
ax1.minorticks_on()

# No Xlabels in Main subplot
ax1.tick_params(axis='x',which='major',labelbottom='off')
ax1.set_ylim(0,2.5)

# Title and legend
ax1.set_title(tsl.keywords['arq_spec'])
ax1.legend()

# Mass
if distance is not None:
    dist_cm = (distance * u.Mpc).to(u.cm)
    mass = tsl.keywords['Mcor_tot'] * flux_units * 4 * pi * dist_cm**2 / L_sun.

# Text with extra info
av   = r'A$_V$ = %.2f mag'       % tsl.keywords['A_V']
v0   = r'v$_*$ = %.1f km/s'      % tsl.keywords['v_0']
vd   = r'$\sigma_*$ = %.1f km/s' % tsl.keywords['v_d']
sn   = 'S/N = %.1f'              % tsl.keywords['SN_normwin']
chi  = r'$\chi^2$ = %.1f'        % tsl.keywords['chi2']
adev = 'adev = %.1f'             % tsl.keywords['adev']
ltext = [av, v0, vd, sn, chi, adev]
# Add mass after "vd" if distance is provided
if distance is not None:
    mass = r'mass = %.2g M$_{\odot}$' % mass.value
    ltext.insert(3, mass)
text = '\n'.join(ltext)

#'figure fraction': 0,0 is lower left of figure and 1,1 is upper right
# 'axes fraction':  0,0 is lower left of axes and 1,1 is upper right
ax1.annotate(text, xy=(0, 1), xytext=(15, -15), fontsize=10,
```

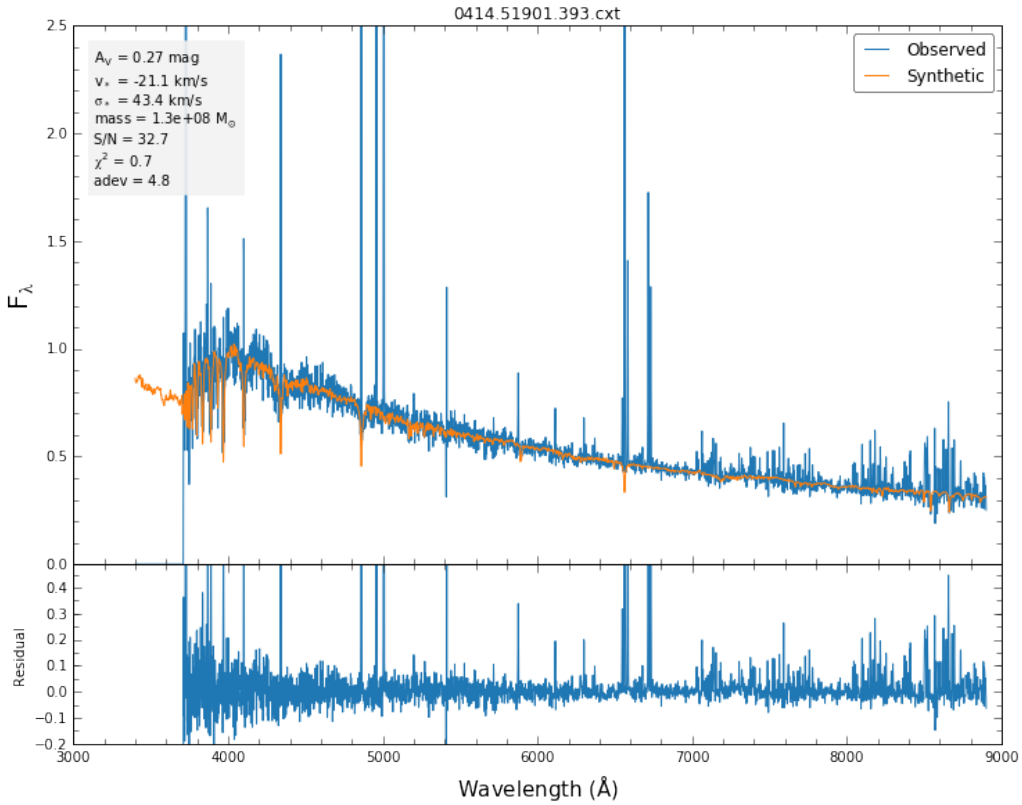
```
            xycoords='axes fraction', textcoords='offset points',
            bbox=dict(facecolor='0.95', alpha=0.9),
            horizontalalignment='left', verticalalignment='top')

        # Residual spectra
        ax2 = plt.subplot(gs[1])
        residual = (tsl.spectra.f_obs - tsl.spectra.f_syn)
        ax2.plot(tsl.spectra.l_obs, residual)

        ax2.minorticks_on()
        #ax2.tick_params(axis='y',which='minor',left='off',right='off')
        minorLocator = AutoMinorLocator(2)
        ax2.yaxis.set_minor_locator(minorLocator)
        ax2.set_ylim(-0.2,0.49)

        # Axis Labes
        ax1.set_ylabel(r'$F_\lambda$', fontsize=20)
        ax2.set_ylabel('Residual')
        ax2.set_xlabel(r'Wavelength ($\AA$)', fontsize=15)
```

```
Out[7]: <matplotlib.text.Text at 0x10e77a550>
```



1.2.2 `Population` Table

The `population` Table contains all the arrays with information regarding the Star Formation History (SFH) estimated from the stellar fitting.

It contains several 8 columns:

```
In [8]: print (tsl.population.columns)

{
popx : type=>f8,
popmu_ini : type=>f8,
popmu_cor : type=>f8,
popage_base : type=>f8,
popZ_base : type=>f8,
popfbase_norm : type=>f8,
popYAV_flag : type=>f8,
popMstars : type=>f8
```

```
}
```

The size of these arrays are the same as the BASE file you have used for the fittings, i.e., the number of SSPs:

```
In [9]: print 'arq_base:', tsl.keywords['arq_base']
        print 'N_base:', tsl.keywords['N_base']
        print 'popx.shape:', tsl.population.popx.shape
```

```
arq_base: Base.BC03.S
N_base: 150
popx.shape: (150,)
```

150	[N_base]					
bc2003_hr_m22_chab_ssp_020.spec	0.00100e9	0.00010	age020_m22	1.0000	0	0.0000
bc2003_hr_m22_chab_ssp_045.spec	0.00316e9	0.00010	age045_m22	1.0000	0	0.0000
bc2003_hr_m22_chab_ssp_055.spec	0.00501e9	0.00010	age055_m22	0.9582	0	0.0000
bc2003_hr_m22_chab_ssp_061.spec	0.00661e9	0.00010	age061_m22	0.9297	0	0.0000
bc2003_hr_m22_chab_ssp_067.spec	0.00871e9	0.00010	age067_m22	0.9030	0	0.0000
bc2003_hr_m22_chab_ssp_070.spec	0.01000e9	0.00010	age070_m22	0.8902	0	0.0000
bc2003_hr_m22_chab_ssp_078.spec	0.01445e9	0.00010	age078_m22	0.8589	0	0.0000
bc2003_hr_m22_chab_ssp_090.spec	0.02512e9	0.00010	age090_m22	0.8174	0	0.0000
bc2003_hr_m22_chab_ssp_104.spec	0.04000e9	0.00010	age104_m22	0.7830	0	0.0000
bc2003_hr_m22_chab_ssp_110.spec	0.05500e9	0.00010	age110_m22	0.7597	0	0.0000
bc2003_hr_m22_chab_ssp_116.spec	0.10152e9	0.00010	age116_m22	0.7138	0	0.0000
bc2003_hr_m22_chab_ssp_120.spec	0.16090e9	0.00010	age120_m22	0.6775	0	0.0000
bc2003_hr_m22_chab_ssp_125.spec	0.28612e9	0.00010	age125_m22	0.6412	0	0.0000
bc2003_hr_m22_chab_ssp_130.spec	0.50880e9	0.00010	age130_m22	0.6083	0	0.0000
bc2003_hr_m22_chab_ssp_135.spec	0.90479e9	0.00010	age135_m22	0.5785	0	0.0000
bc2003_hr_m22_chab_ssp_138.spec	1.27805e9	0.00010	age138_m22	0.5588	0	0.0000
bc2003_hr_m22_chab_ssp_139.spec	1.43400e9	0.00010	age139_m22	0.5532	0	0.0000
bc2003_hr_m22_chab_ssp_150.spec	2.50000e9	0.00010	age150_m22	0.5296	0	0.0000
bc2003_hr_m22_chab_ssp_158.spec	4.25000e9	0.00010	age158_m22	0.5110	0	0.0000
bc2003_hr_m22_chab_ssp_166.spec	6.25000e9	0.00010	age166_m22	0.4985	0	0.0000
bc2003_hr_m22_chab_ssp_171.spec	7.50000e9	0.00010	age171_m22	0.4931	0	0.0000
bc2003_hr_m22_chab_ssp_181.spec	10.00000e9	0.00010	age181_m22	0.4854	0	0.0000
bc2003_hr_m22_chab_ssp_193.spec	13.00000e9	0.00010	age193_m22	0.4791	0	0.0000
bc2003_hr_m22_chab_ssp_201.spec	15.00000e9	0.00010	age201_m22	0.4766	0	0.0000
bc2003_hr_m22_chab_ssp_213.spec	18.00000e9	0.00010	age213_m22	0.4735	0	0.0000
bc2003_hr_m32_chab_ssp_020.spec	0.00100e9	0.00040	age020_m32	1.0000	0	0.0000
bc2003_hr_m32_chab_ssp_045.spec	0.00316e9	0.00040	age045_m32	1.0000	0	0.0000
bc2003_hr_m32_chab_ssp_055.spec	0.00501e9	0.00040	age055_m32	0.9532	0	0.0000
bc2003_hr_m32_chab_ssp_061.spec	0.00661e9	0.00040	age061_m32	0.9243	0	0.0000
bc2003_hr_m32_chab_ssp_067.spec	0.00871e9	0.00040	age067_m32	0.9001	0	0.0000

The order is the same as in the `arq_base` file.
`popage_base` and `popZ_base` are the *age* and *metallicity* arrays of each element.

```
In [10]: # Print the first 30 elements
         print tsl.population.popZ_base[:30]
```

```
[  9.99999975e-05   9.99999975e-05   9.99999975e-05   9.99999975e-05
   9.99999975e-05   9.99999975e-05   9.99999975e-05   9.99999975e-05
   9.99999975e-05   9.99999975e-05   9.99999975e-05   9.99999975e-05
   9.99999975e-05   9.99999975e-05   9.99999975e-05   9.99999975e-05
   9.99999975e-05   9.99999975e-05   9.99999975e-05   9.99999975e-05
   9.99999975e-05   9.99999975e-05   9.99999975e-05   9.99999975e-05
```

```
   9.99999975e-05   3.99999990e-04   3.99999990e-04   3.99999990e-04
   3.99999990e-04   3.99999990e-04]
```

```
In [11]: print tsl.population.popage_base[:30]

[  1.00000000e+06   3.16000000e+06   5.01000000e+06   6.61000000e+06
   8.71000000e+06   1.00000000e+07   1.44500000e+07   2.51200000e+07
   4.00000000e+07   5.50000000e+07   1.01520000e+08   1.60900000e+08
   2.86120000e+08   5.08800000e+08   9.04790016e+08   1.27805005e+09
   1.43400000e+09   2.50000000e+09   4.24999987e+09   6.24999987e+09
   7.50000026e+09   1.00000000e+10   1.29999995e+10   1.50000005e+10
   1.79999990e+10   1.00000000e+06   3.16000000e+06   5.01000000e+06
   6.61000000e+06   8.71000000e+06]
```

As you can see, values are repeated. You can obtained the total number of unique ages and metallicities using `np.unique`:

```
In [12]: import numpy as np

         # You can access also the columns like in an astropy Table
         ageBase = np.unique(tsl.population['popage_base'])
         zBase   = np.unique(tsl.population['popZ_base'])

         print 'Age (%i):' % ageBase.size, ageBase
         print 'Z (%i)' % zBase.size, zBase
```

```
Age (25): [  1.00000000e+06   3.16000000e+06   5.01000000e+06   6.61000000e+06
   8.71000000e+06   1.00000000e+07   1.44500000e+07   2.51200000e+07
   4.00000000e+07   5.50000000e+07   1.01520000e+08   1.60900000e+08
   2.86120000e+08   5.08800000e+08   9.04790016e+08   1.27805005e+09
   1.43400000e+09   2.50000000e+09   4.24999987e+09   6.24999987e+09
   7.50000026e+09   1.00000000e+10   1.29999995e+10   1.50000005e+10
   1.79999990e+10]
Z (6) [  9.99999975e-05   3.99999990e-04   4.00000019e-03   8.00000038e-03
   1.99999996e-02   5.00000007e-02]
```

If you define $Z_{\odot} = 0.02$, you can create a new metallicity array in solar units:

```
In [13]: zsun = zBase[4] # Solar metallicity is the 5th element (in this BASE!!)
         zsunBase = zBase / zsun
         print zsunBase
```

```
[ 0.005       0.02        0.20000001  0.40000003  1.          2.50000009]
```

You now can have a look at the SFH results.

The two arrays that your are going to look at are `popx` (light vector) and `popmu_cor` (mass vector).

`popx` will give you information about what populations is given a fraction of the total light of the spectrum, and `popmu_cor` the fraction of the mass of the total.

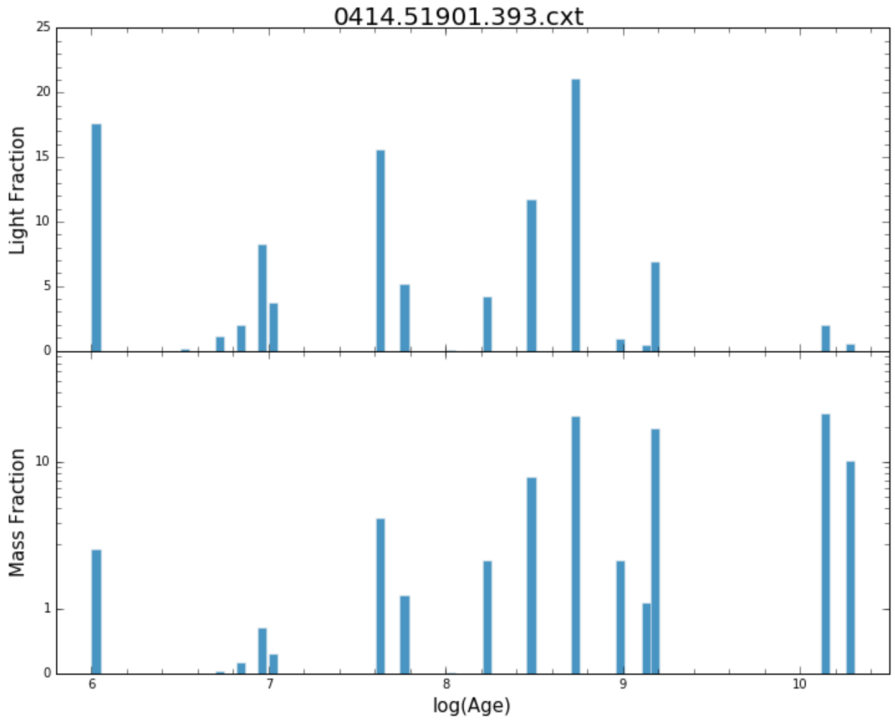
Exercice

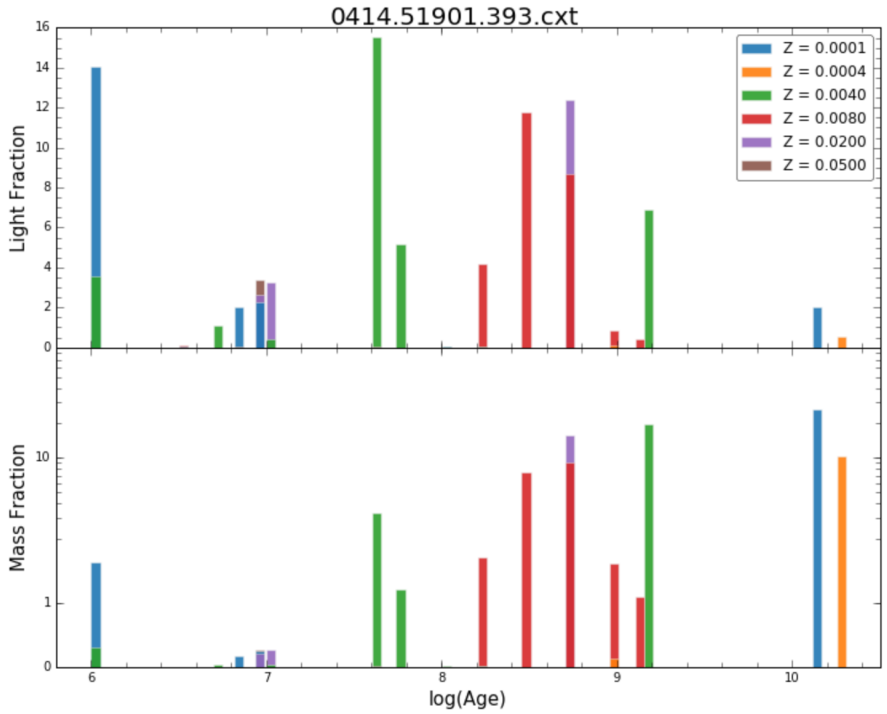
Make a figure with two subplots (equal size): * one for the light SFH (`popx`) * second the for the mass (`popmu_cor`) * Make a option to plot in "collapse" or "discrete" mode (each metallicity with a different color and label) * Plot log(Age) in x-axis

Hint 1: Use a bar plot (matplotlib's `bar`) *Hint 2*: For the discrete mode take into account height of each metallicity bar (`zorder` in matplotlib's `bar`)

** Solution **

Collapsed




0414.51901.393.cxt
Z = 0.0001
Z = 0.0004
Z = 0.0040
Z = 0.0080
Z = 0.0200
Z = 0.0500
Light Fraction
Mass Fraction
log(Age)

Discrete

```
In [14]: from matplotlib.ticker import ScalarFormatter
         import matplotlib.pyplot as plt
         from matplotlib import gridspec
         import numpy as np
         import matplotlib

         def sortHistBars(lpatch):
             x = np.array([p._x for p in lpatch])
             y = np.array([p._height for p in lpatch])
             zor = np.lexsort((y,x))
             d = {}
             for i,(ix,iy) in enumerate(zip(x[zor],y[zor])):
                 d[(ix,iy)] = -i
             [p.set_zorder(d[(p._x,p._height)]) for p in lpatch]

         collapse = False
         cumsum   = False

         zcolor = [item['color'] for item in matplotlib.rcParams['axes.prop_cycle']]

         fig = plt.figure(figsize=(10,8))
         gs = gridspec.GridSpec(2, 1)
         gs.update(left=0.05, bottom=0.05, right=0.98, hspace=0.0)

         ageBase = np.unique(tsl.population['popage_base'])
         zBase   = np.unique(tsl.population['popZ_base'])

         lageBase = np.log10(ageBase)
         log_age = np.log10(tsl.population['popage_base'])

         # Bin width
         d = log_age[1:] - log_age[:-1]
         Bwidth = np.min(d[d>0])
         #Bwidth  = np.append(np.diff(lageBase), np.diff(lageBase)[-1])

         axl = plt.subplot(gs[0]) # Light
         axm = plt.subplot(gs[1]) # Mass

         patch_light = []
         patch_mass = []
         light_cum = np.zeros(lageBase.shape)
         mass_cum  = np.zeros(lageBase.shape)

         for z,zcol in zip(zBase,zcolor):
```

```
            idz = tsl.population['popZ_base'] == z
            lbottom = light_cum if cumsum else None
            mbottom = mass_cum  if cumsum else None
            if not collapse:
                pl = axl.bar(log_age[idz], tsl.population['popx'][idz], label='Z = %.4
                patch_light.extend(pl)
                pm = axm.bar(log_age[idz], tsl.population['popmu_cor'][idz], label='Z
                patch_mass.extend(pm)
            light_cum += tsl.population['popx'][idz]
            mass_cum  += tsl.population['popmu_cor'][idz]

        if collapse:
            pl = axl.bar(lageBase, light_cum, width=Bwidth, alpha=0.9)
            patch_light.extend(pl)
            pm = axm.bar(lageBase, mass_cum, width=Bwidth, alpha=0.9)
            patch_mass.extend(pm)

        # Order the height of bars
        if not cumsum:
            sortHistBars(patch_light)
            sortHistBars(patch_mass)

        axl.set_ylabel('Light Fraction', fontsize=15)
        axm.set_ylabel('Mass Fraction', fontsize=15)
        axm.set_xlabel('log(Age)', fontsize=15)

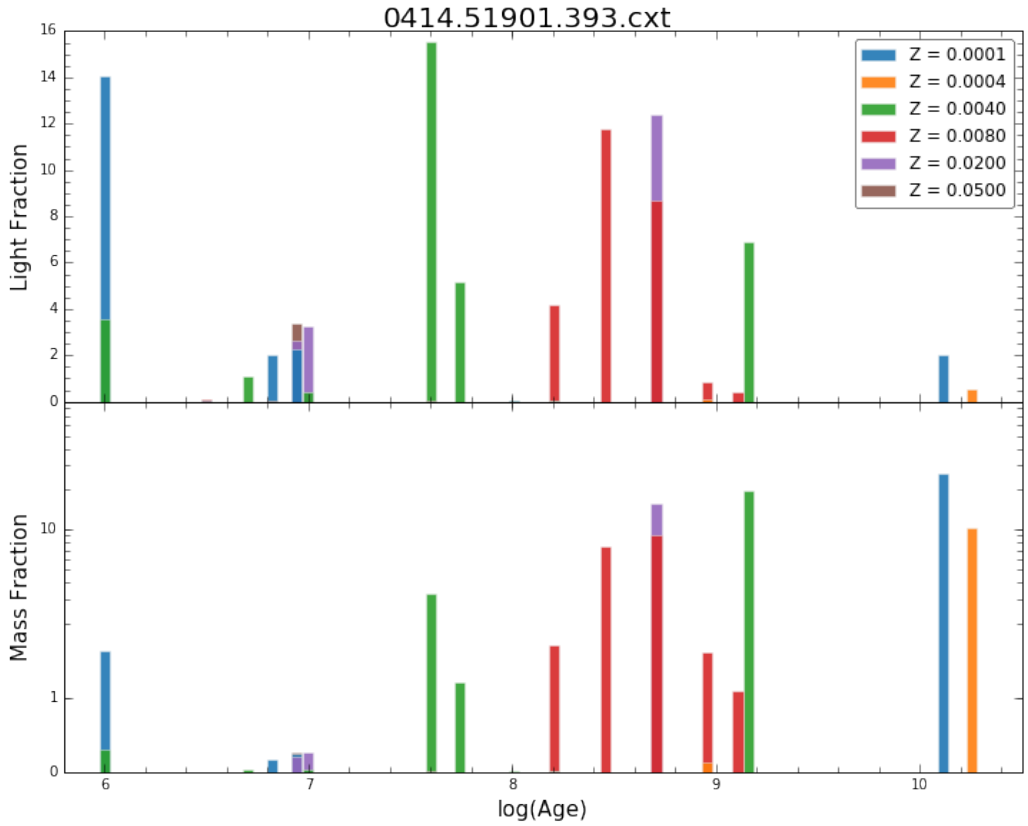
        if not collapse:
            axl.legend()

        axl.minorticks_on()
        ax1.tick_params(axis='x',which='major',labelbottom='off')
        axm.minorticks_on()

        axl.set_xlim(5.8,10.5)
        axm.set_xlim(5.8,10.5)
        axm.set_ylim(0.,90.)
        axm.set_yscale('symlog',subsy=np.arange(1,10))
        axm.yaxis.set_major_formatter(ScalarFormatter())

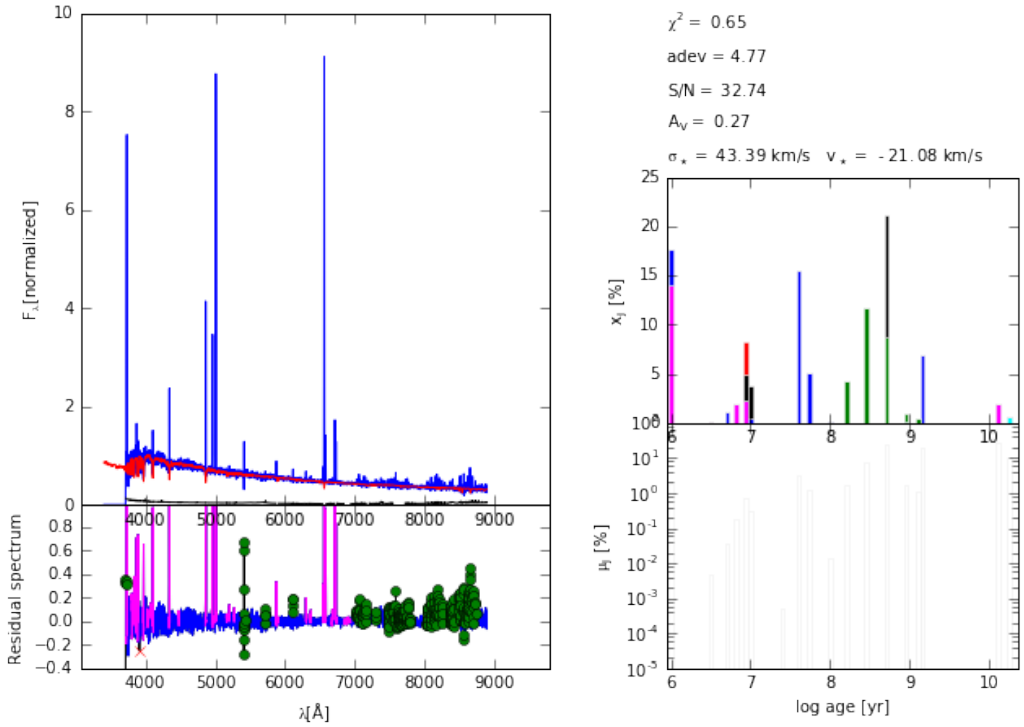
        fig.suptitle(tsl.keywords['arq_spec'], fontsize=20)
```

Out[14]: <matplotlib.text.Text at 0x112541390>




```
In [15]: from pystarlight.plots.plotstarlightfits import plot_fits_and_SFH

         psfh = plot_fits_and_SFH(tsl)
         psfh.plot_fig_starlight()
```



1.3 Weighted average properties

Let's obtain some average properties as age or metallicity.

If you remember STARLIGHT manual, the total sum of x_j vectors from the SFH solution might be large than 100%.

```
In [16]: print tsl.population['popx'].sum()
```

```
101.440400113
```

```
In [17]: print tsl.population['popmu_cor'].sum()
```

```
100.000996926
```

Thus, we need to normalize these vectors if we want to use them to obtain them as weights for estimating mean properties:

```
In [18]: popx_norm = tsl.population['popx'] / tsl.population['popx'].sum()
         print popx_norm.sum()
```

```
1.0
```

$$\langle log\, t \rangle_L = \sum_{t,Z} x_{tZ} \times log\, t$$

Exercise: Calculate the light weighted age

```
In [19]: popx_norm = tsl.population['popx'] / tsl.population['popx'].sum()
         log_age = np.log10(tsl.population['popage_base'])

         print (log_age * popx_norm).sum()
```

```
7.77097316799
```

```
In [20]: rpopx = tsl.population['popx'].reshape([zBase.size, ageBase.size]).T
         print rpopx.shape
```

```
(25, 6)
```

```
In [21]: popx__t = rpopx.sum(axis=1)
         print popx__t.shape
```

```
(25,)
```

```
In [22]: popx__t_norm = popx__t / rpopx.sum()
         print np.tensordot(popx__t_norm, lageBase, (0, 0))
```

```
7.77097316799
```

$$\langle log\, Z \rangle_M = \sum_{t,Z} m_{tZ} \times log\, Z$$

Exercise: Calculate the mass weighted metallicity

```
In [23]: popmu_cor_norm = tsl.population['popmu_cor'] / tsl.population['popmu_cor'].sum
         print popmu_cor_norm.sum()
```

```
1.0
```

```
In [24]: log_z = np.log10(tsl.population['popZ_base'])
         w_z = (log_z * popmu_cor_norm).sum()
         print w_z, 10**w_z
```

```
-2.77632594823 0.00167368626361
```

```
In [25]: rpopmu_cor = tsl.population['popmu_cor'].reshape([zBase.size, ageBase.size]).T
         popmu_cor__Z = rpopmu_cor.sum(axis=0)
         popmu_cor__Z_norm = popmu_cor__Z / rpopmu_cor.sum()

         print np.tensordot(popmu_cor__Z_norm, np.log10(zBase), (0, 0))
```

```
-2.77632594823
```

Exercise: Calculate the *MASS* weighted age

$$\langle log\, t \rangle_M = \sum_{t,Z} m_{tZ} \times log\, t$$

```
In [26]: print (log_age * popmu_cor_norm).sum()
```

```
9.18050776543
```

1.3.1 Fraction of young, intermediate and old populations: x_Y, x_I, x_O

Population synthesis studies in the past found that a useful way to summarize the SFH is to condense the age distribution encoded in the population vector into age ranges.

This strategy comes from a time when the analysis was based on equivalent widths and colors.

Exercice

Let's divide the light fraction into 3 components:

- Young (x_Y): t $\leq$ 0.14 Gyr
- Intermediate (x_I): 0.14 < t $\leq$ 1.4 Gyr
- Old populations (x_O): t > 1.4 Gyr

This represents the contribution of young, intermediate age and old (>1.4 Gyr) stars to the observed light at the normalization window.

Hint: Use `popx_norm`

** Solution: **

```
In [27]: popx_norm = tsl.population['popx'] / tsl.population['popx'].sum()

         tY = 0.14e9
         tI = 1.4e9
         age_base = tsl.population['popage_base']

         itY = age_base <= tY
```

```
        itI = np.bitwise_and(age_base > tY, age_base <= tI)
        itO = age_base > tI

        Ypopx = popx_norm[itY].sum()
        Ipopx = popx_norm[itI].sum()
        Opopx = popx_norm[itO].sum()

        print 'Fraction Young: %.2f' % Ypopx
        print 'Fraction Inter: %.2f' % Ipopx
        print 'Fraction   Old: %.2f' % Opopx
        print 'Sum           : %.2f' % (Ypopx + Ipopx + Opopx)
```

```
Fraction Young: 0.53
Fraction Inter: 0.38
Fraction   Old: 0.09
Sum           : 1.00
```

Exercice

Do the same for the mass fraction (`popmu_cor_norm`)
** Solution: **

```
In [28]: popmu_cor_norm = tsl.population['popmu_cor'] / tsl.population['popmu_cor'].sum

         tY = 0.14e9
         tI = 1.4e9
         age_base = tsl.population['popage_base']

         itY = age_base <= tY
         itI = np.bitwise_and(age_base > tY, age_base <= tI)
         itO = age_base > tI

         Ypopmu_cor = popmu_cor_norm[itY].sum()
         Ipopmu_cor = popmu_cor_norm[itI].sum()
         Opopmu_cor = popmu_cor_norm[itO].sum()

         print 'Fraction Young: %.2f' % Ypopmu_cor
         print 'Fraction Inter: %.2f' % Ipopmu_cor
         print 'Fraction   Old: %.2f' % Opopmu_cor
         print 'Sum           : %.2f' % (Ypopmu_cor + Ipopmu_cor + Opopmu_cor)
```

```
Fraction Young: 0.08
Fraction Inter: 0.37
Fraction   Old: 0.56
Sum           : 1.00
```

1.4 Estimating the Star Formation Rate (SFR)

The base used in our fits comprises instantaneous bursts (i.e., SSPs).

Thus, despite the large number of ages considered, our SFHs are not continuous and hence not derivable.

Nonetheless, there are ways of defining star formation rates (SFR).

The simplest way to estimate a SFR is to cumulate all the stellar mass formed since a lookback time of tSF and perform a mass-over-time average:

$$\overline{\mathcal{SFR}}(t_{SF}) = \frac{1}{t_{SF}} \sum_{t<t_{SF}} \mathcal{M}'_t$$

where $\mathcal{M}'_t$ is the mass (in general, per unit area) of stars formed at lookback time t.

This equation gives the mean SFR surface density since t = t_{SF}.

One can tune t_{SF} to reach different depths in the past, but as t_{SF} increases this estimator becomes increasingly useless, converging to the mass density divided by $t_\infty \equiv 14$ Gyr (the largest age in the base).

The stellar mass in this equation differs from the definitions of $\langle \log t \rangle_M$ and $\langle \log Z \rangle_M$.

The latter (`popmu_cor`) are corrected by the mass returned to the medium by stars during their evolution (thus reflecting the mass currently in stars), while in the computation of SFRs this correction should not be applied (`popmu_ini`).

Exercise

Calculate the SFR for t_{SF} = 100 Myr

$$\overline{\mathcal{SFR}}(t_{SF}) = \frac{1}{t_{SF}} \sum_{t<t_{SF}} \mathcal{M}'_t$$

Assume a distance of 126 Mpc and flux_units = 1e-17

Hint 1 : Normalize popmu_ini to unity and convert it to mass

```
In [34]: from astropy.constants import L_sun
         import astropy.units as u
         from math import pi

         tsf = 0.1e9 # 100 Myr

         distance = 126.
         flux_units = 1e-17 * u.erg / u.cm**2 / u.s / u.A

         dist_cm = (distance * u.Mpc).to(u.cm)
         Mini_tot = tsl.keywords['Mini_tot'] * flux_units *\
                     4 * pi * dist_cm**2 / L_sun.cgs

         Mini = (tsl.population.popmu_ini / 100.0) * Mini_tot.value

         # Select ages
```

```
        itsf = tsl.population.popage_base < tsf

        # Total Mass in tsf
        Mini_tsf = Mini[itsf].sum()

        # SFR in tsf
        SFR_tsf = Mini_tsf / tsf # M_sun / yr

        print 'SFR (tSF=%.1f Gyr) = %.3f M_sun/yr' % (tsf/1.e9, SFR_tsf)
```

```
SFR (tSF=0.1 Gyr) = 0.113 M_sun/yr
```

It is often useful to consider SFRs in relation to some fiducial value, instead of absolute units.

The classical example is Scalo's birthrate parameter, b, which measures the SFR in the recent past ($t < t_{SF}$) with respect to its average over the whole lifetime of the system.

This is trivially obtained dividing $\overline{\mathcal{SFR}}(t_{SF})$ by its asymptotic value

$$b(t_{SF}) = \frac{\overline{\mathcal{SFR}}(t_{SF})}{\overline{\mathcal{SFR}}(t_{\infty})} = \frac{t_{\infty}}{t_{SF}} \frac{\sum_{t<t_{SF}} \mathcal{M}'}{\mathcal{M}'}$$

Exercice

Estimate the Scalor's birthrate parameter b for t_{SF} = 100 Myr

```
In [30]: from astropy.constants import L_sun
         import astropy.units as u
         from math import pi

         tsf = 0.1e9 # 100 Myr

         distance = 126.
         flux_units = 1e-17 * u.erg / u.cm**2 / u.s / u.A

         dist_cm = (distance * u.Mpc).to(u.cm)
         Mini_tot = tsl.keywords['Mini_tot'] * flux_units *\
                     4 * pi * dist_cm**2 / L_sun.cgs

         Mini = (tsl.population.popmu_ini / 100.0) * Mini_tot.value

         # Select ages
         itsf = tsl.population.popage_base < tsf

         # Total Mass in tsf
         Mini_tsf = Mini[itsf].sum()

         # SFR in tsf
```

```
        SFR_tsf = Mini_tsf / tsf # M_sun / yr

        # Averate SFR
        SFR_average = Mini.sum() / tsl.population.popage_base.max()

        # Scalo b parameter
        b_scalo = SFR_tsf / SFR_average

        print 'b_Scalo (tSF=%.1f Gyr) = %.3f' % (tsf/1.e9, b_scalo)
```

```
b_Scalo (tSF=0.1 Gyr) = 9.304
```

1.5 StarlightUtils

If you need to rebin a spectra (pre-precessing step for `STARLIGHT`), you can use `ReSamplingMatrixNonUniform`:

```
In [31]: from pystarlight.util.StarlightUtils import ReSamplingMatrixNonUniform as RSMN
         import numpy as np

         lorig = np.linspace(3400, 8900, 9000) * 1.001
         lresam = np.linspace(3400, 8900, 5000)

         forig = np.random.normal(size=len(lorig))**2

         matrix = RSMNU(lorig, lresam)
         fresam = np.dot(matrix, forig)

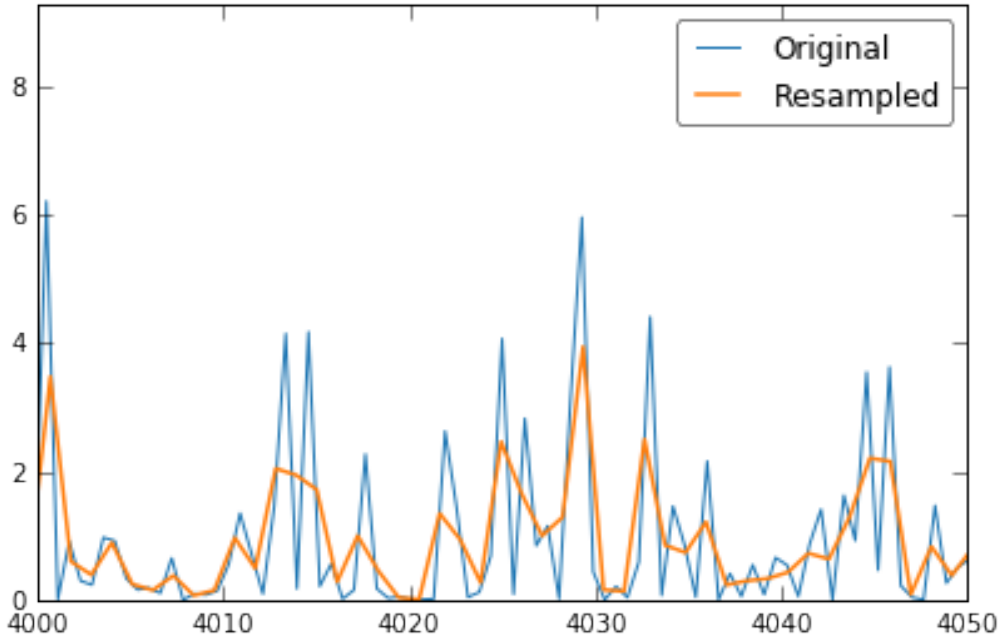
         print np.trapz(forig, lorig), np.trapz(fresam, lresam)

         npfresam = np.interp(lresam, lorig, forig)
         print np.trapz(npfresam, lresam)

         plt.plot(lorig, forig, label='Original')
         plt.plot(lresam, fresam, lw=1.5, label='Resampled')
         plt.xlim(4000, 4050)
         plt.ylim(0, forig.max()*0.5)
         plt.legend()
```

```
5510.26816333 5500.44402525
5472.2955774
```

```
Out[31]: <matplotlib.legend.Legend at 0x12b328610>
```


Original
Resampled
8
6
4
2
0
4000
4010
4020
4030
4040
4050